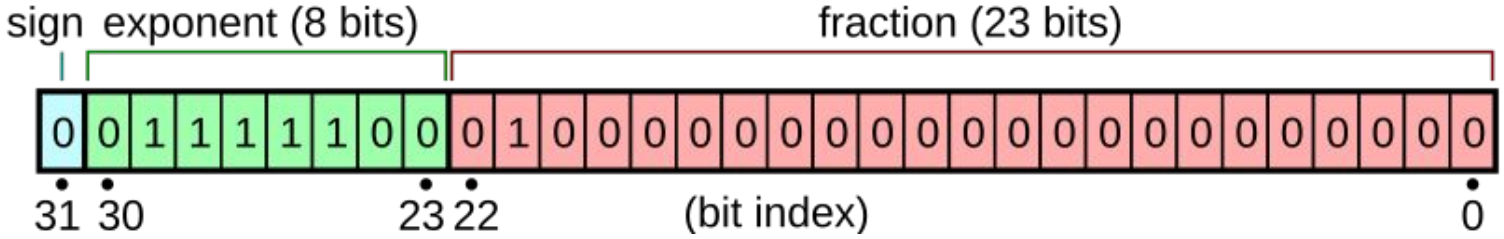
Сопроцессор x87

Повторение: IEEE754



- Знак, экспонента, мантисса
- Смещение экспоненты
- Нормализованные и денормализованные числа
- +inf, -inf, NaN

x87

- Раньше операции с fp значениями выполнялись через отдельный математический сопроцессор
- Сопроцессоры имели номера, оканчивающиеся на 87, поэтому соответствующий набор инструкций был назван x87
- Современные компиляторы чаще используют SSE/AVX

x87

- У x87 есть 8 регистров: ST0 – ST7
- Но, устройство регистров x87 отличается от привычных нам регистров. Это кольцевой стек.
- Имена ST0, ST1, … это не фиксированные физические регистры, а относительные имена от текущей вершины стека.
- ST0 всегда означает текущую вершину, а не регистр с номером 0.

x87 (Stack overflow/underflow)

- Что произойдет, если переполнить стек (или достать значение из пустого стека)?

x87 (Stack overflow/underflow)

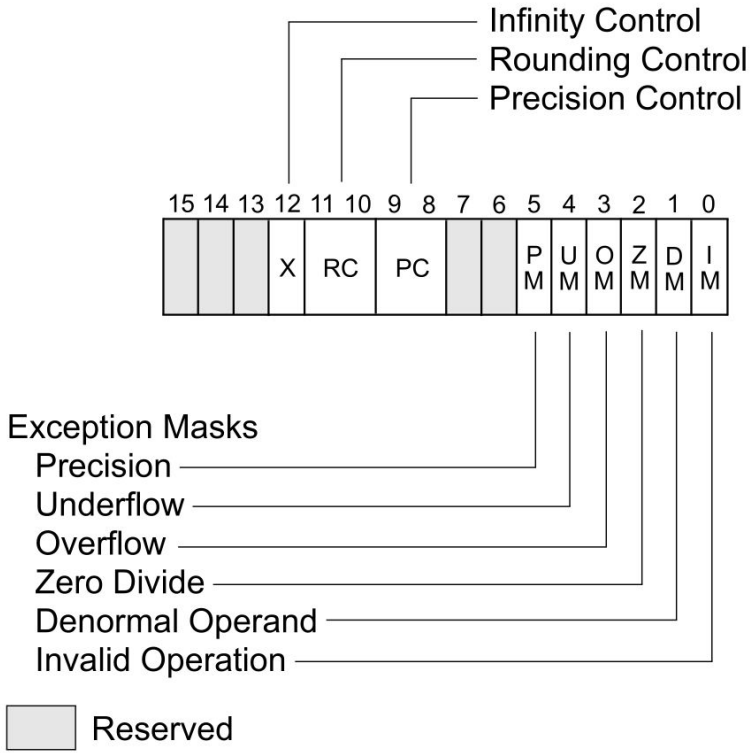
- Что произойдет, если переполнить стек (или достать значение из пустого стека)?
- Ответ на этот вопрос довольно… комплексный

x87

- В GDB: fctrl
- В SDM:

  8.1.5 x87 FPU Control Word
- Управляет масками исключений,

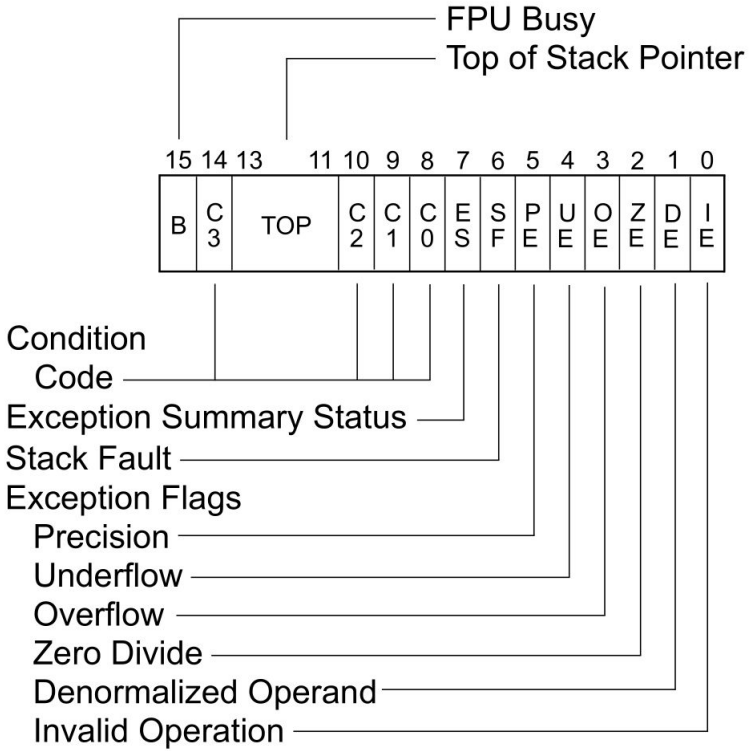
  точностью и режимом округления



x87

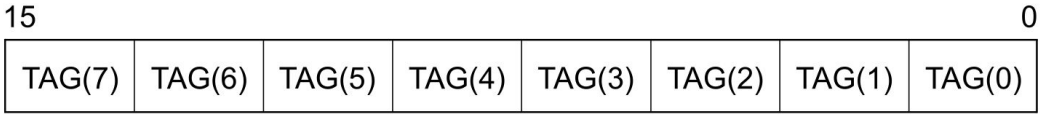
- В GDB: fstat
- В SDM:

  8.1.3 x87 FPU Status Register
- Показывает текущее состояние FPU



x87

- В GDB: ftag
- В SDM: 8.1.7 x87 FPU Tag Word
- Описывает состояния каждого физического x87 регистра



TAG Values

00 — Valid
01 — Zero
10 — Special: invalid (NaN, unsupported), infinity, or denormal
11 — Empty

x87. Специальные инструкции

- `finit` : сбрасывает состояние x87 (обнуляет регистры, сбрасывает значения Control Word, Status Register, Tag Word)
- `fwait` : явно синхронизирует исключения (в x87 они могут быть отложенными и “вылететь” не на проблемной инструкции, а на последующей)
- `fclex/fnclex` : очищает флаги ошибок

x87. Загрузка и сохранение чисел

- `fld` : загрузить fp значение в стек
- `fst/fstp` : сохранить значение ST0 в память/сохранить + pop
- `fild` : загрузить целое число и преобразовать в fp
- `fist/fistp` : сохранить значение ST0 в память как целое/сохранить + pop
- `fldz` : загрузить 0
- `fld1` : загрузить 1
- `fldpi` : загрузить ?

x87. Загрузка и сохранение чисел

- `fld` : загрузить fp значение в стек
- `fst/fstp` : сохранить значение ST(0) в память/сохранить + pop
- `fild` : загрузить целое число и преобразовать в fp
- `fist/fistp` : сохранить значение ST(0) в память как целое/сохранить + pop
- `fldz` : загрузить 0
- `fld1` : загрузить 1
- `fldpi` : загрузить π

x87. Арифметические операции

- Инструкции x87 могут работать только с регистрами x87 и памятью
- Если операнды инструкции – регистры, то один из них всегда должен быть ST0
- Если операнд – память, то операция выполняется с регистром ST0, результат записывается в ST0

x87. Арифметические операции

- `fadd` : сложение
- `faddp` : сложение + pop
- `fiadd` : сложение с целым числом из памяти
- `fmul` : умножение
- `fmulp` : умножение + pop
- `fimul` : умножение на целое число из памяти

x87. Арифметические операции

- `fsub st0 ,st1` : вычитание, ST0 = ST0 - ST1
- `fsubp` : вычитание + pop
- `fsubr st0 ,st1` : reverse-вычитание, ST0 = ST1 - ST0
- `fsubrp` : reverse-вычитание + pop
- `fisub/fisubr` : вычитание/reverse-вычитание с целым числом из памяти
- `fdiv st0 ,st1` : деление, ST0 = ST0 / ST1
- `fdivp` : деление + pop
- `fdivr st0 ,st1` : reverse-деление, ST0 = ST1 / ST0
- `fdivrp` : reverse-деление + pop
- `fidiv/fidivr` : деление/reverse-деление с целым числом из памяти

x87. Сравнение чисел

Устаревшие инструкции:

- `fcom/fucom` : сравнить ST0 и ST1, выставить флаги в Status Word
- `fcomp/fucomp` : сравнить + pop
- `fcompp/fucompp` : сравнить + pop + pop (т.е. pop ST0 и ST1)

(Инструкции без u могут сгенерировать Invalid Op. при сравнении с NaN)

Использование:

```
1 fcompp
2 fnstsw  ax
3 sahf
4
5 ja      greater
6 jb      less
7 je      equal
8 jp      unordered
```

x87. Сравнение чисел

Современные инструкции:

- `fcomi/fucomi` : сравнить ST0 и ST1, выставить флаги EFLAGS
- `fcomip/fucomip` : сравнить + pop

(Инструкции без u могут сгенерировать Invalid Op. при сравнении с NaN)

Использование:

```
1 fcomi
2
3 ja      greater
4 jb      less
5 je      equal
6 jp      unordered
```

x87. Сравнение чисел

- Используются unsigned-переходы: `ja, jb`

- Новый переход: `jp` (для проверки на сравнение с NaN)

- Для сравнения на равенство не используем `je`!!!!
  Только через сравнение с eps